

Algoritmo de Alocação via Otimizador PL v2.0

Sistema SIVIRA - Otimização de Produção

Documentação Técnica

Dezembro 2025

Sumário

1	Introdução	2
1.1	Objetivos do Sistema	2
2	Arquitetura do Sistema	2
2.1	Componentes	2
3	Formulação Matemática	2
3.1	Variáveis de Decisão	2
3.2	Função Objetivo	3
3.3	Restrições	3
3.3.1	Restrição 1: Unicidade por Pedido	3
3.3.2	Restrição 2: Permutação Única (Ordenação)	3
3.3.3	Restrição 3: Tempo Máximo de Espera (Gap Zero)	3
3.3.4	Restrição 4: Conflitos Temporais (Equipamentos)	4
3.3.5	Restrição 5: Fim Obrigatório (Deadline Rígido)	4
4	Backward Scheduling	4
4.1	Algoritmo	4
4.2	Representação Visual	4
5	Complexidade Computacional	4
6	Comparação: Sistema Sequencial vs PL v2.0	5
7	Implementação	5
7.1	Exemplo de Uso	5
7.2	Parâmetros de Configuração	5
8	Conclusão	5

1 Introducao

O Otimizador PL v2.0 e um sistema de otimizacao de producao que utiliza **Programacao Linear Inteira Mista (MILP)** para determinar a ordem otima de execucao de pedidos em uma linha de producao.

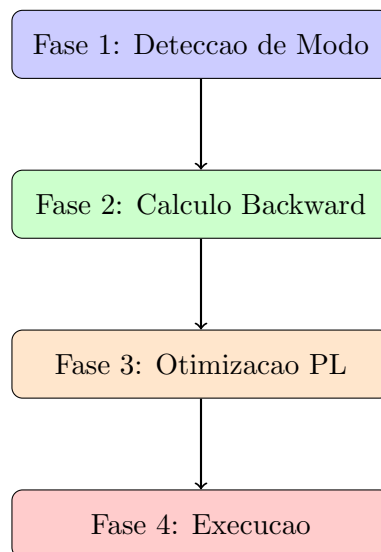
O sistema foi desenvolvido utilizando o solver **OR-Tools CP-SAT** do Google, que oferece alta performance para problemas de satisfacao de restricoes e otimizacao combinatoria.

1.1 Objetivos do Sistema

- Maximizar o numero de pedidos atendidos dentro dos prazos (deadlines)
- Respeitar restricoes de equipamentos (capacidade finita)
- Garantir sequenciamento correto de atividades (gaps temporais)
- Otimizar a ordem de execucao considerando urgencia dos pedidos

2 Arquitetura do Sistema

O sistema e composto por 4 fases principais:



2.1 Componentes

3 Formulacao Matematica

3.1 Variaveis de Decisao

O modelo utiliza variaveis binarias para representar a selecao de pedidos e suas posicoes:

$$x_{p,j} \in \{0, 1\} \tag{1}$$

Onde:

Modulo	Funcao
<code>detector_modo.py</code>	Detecta modo de otimizacao
<code>calculador_horarios_deterministicos.py</code>	Calculo backward scheduling
<code>modelo_pl_ordenacao.py</code>	Modelo PL com OR-Tools CP-SAT
<code>aplicador_ordenacao.py</code>	Executa pedidos na ordem otimizada
<code>executor_unificado.py</code>	Orquestrador principal
<code>executor_v2.py</code>	Adaptador de compatibilidade

Tabela 1: Modulos do sistema PL v2.0

- p = indice do pedido ($p \in P$)
- j = indice da janela temporal candidata ($j \in J_p$)
- $x_{p,j} = 1$ significa que o pedido p sera executado na janela j

Para o modelo de ordenacao, tambem sao utilizadas:

$$pos_i \in \{0, 1, \dots, n-1\} \quad (2)$$

Onde pos_i representa a posicao do pedido i na ordem de execucao.

3.2 Funcao Objetivo

O objetivo e maximizar o numero total de pedidos atendidos:

$$\max \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \quad (3)$$

3.3 Restricoes

3.3.1 Restricao 1: Unicidade por Pedido

Cada pedido utiliza no maximo uma janela temporal:

$$\sum_{j \in J_p} x_{p,j} \leq 1, \quad \forall p \in P \quad (4)$$

3.3.2 Restricao 2: Permutacao Unica (Ordenacao)

Para o modelo de ordenacao, todas as posicoes devem ser diferentes:

$$pos_i \neq pos_j, \quad \forall i \neq j \quad (5)$$

Esta restricao e implementada usando a constraint `AddAllDifferent` do OR-Tools.

3.3.3 Restricao 3: Tempo Maximo de Espera (Gap Zero)

Para atividades consecutivas com tempo maximo de espera igual a zero:

$$t_{fim}^{(a)} = t_{inicio}^{(a+1)} \quad (6)$$

Isso garante que as atividades sejam contiguas, sem intervalo entre elas.

3.3.4 Restricao 4: Conflitos Temporais (Equipamentos)

Se duas janelas se sobrepõem no tempo, não podem ser selecionadas simultaneamente:

$$x_{p_1, j_1} + x_{p_2, j_2} \leq 1 \quad (7)$$

Para todo par $(p_1, j_1), (p_2, j_2)$ onde:

$$[t_1^{ini}, t_1^{fim}] \cap [t_2^{ini}, t_2^{fim}] \neq \emptyset \quad (8)$$

3.3.5 Restricao 5: Fim Obrigatorio (Deadline Rigido)

Para pedidos com deadline obrigatorio:

$$x_{p,j} = 0, \quad \forall j : |t_j^{fim} - d_p| > \epsilon \quad (9)$$

Onde d_p é o deadline do pedido p e $\epsilon = 1$ minuto (tolerancia).

4 Backward Scheduling

Para o cenário determinístico ($\tau_{max} = 0$), o sistema utiliza **backward scheduling** para calcular horários fixos:

4.1 Algoritmo

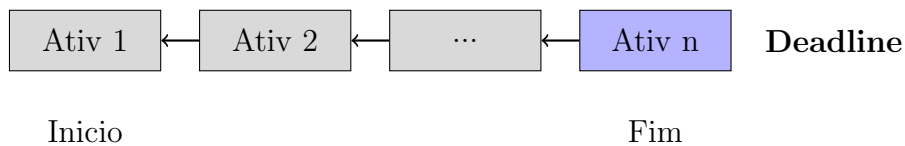
Algorithm 1 Backward Scheduling

```

1:  $t_{atual} \leftarrow deadline$ 
2: for cada atividade  $a$  em ordem reversa do
3:    $t_{fim}^{(a)} \leftarrow t_{atual}$ 
4:    $t_{inicio}^{(a)} \leftarrow t_{fim}^{(a)} - duracao_a$ 
5:    $t_{atual} \leftarrow t_{inicio}^{(a)}$ 
6: end for

```

4.2 Representacao Visual



A última atividade termina exatamente no deadline, e as anteriores são calculadas retroativamente.

5 Complexidade Computacional

O solver OR-Tools CP-SAT utiliza técnicas avançadas de propagação de restrições e busca para encontrar soluções ótimas ou viáveis dentro do timeout configurado (padrão: 30 segundos para ordenação, 600 segundos para modelo completo).

Componente	Complexidade
Variaveis de decisao	$O(n \cdot k)$
Restricoes de unicidade	$O(n)$
Restricoes de conflito	$O(n^2 \cdot k^2)$ (pior caso)

Tabela 2: Complexidade por componente (n = pedidos, k = janelas/pedido)

6 Comparacao: Sistema Sequencial vs PL v2.0

Aspecto	Sequencial	PL v2.0
Ordem de execucao	Fixa (1, 2, 3...)	Otimizada por deadline
Conflitos de equipamentos	Ignora	Modela explicitamente
Gaps temporais	Nao verifica	Restricao $\tau_{max} = 0$
Funcao objetivo	Nenhuma	Maximizar pedidos
Metodo de solucao	Greedy	MILP (CP-SAT)

Tabela 3: Comparacao entre metodos

7 Implementacao

7.1 Exemplo de Uso

```
from otimizador_v2 import ExecutorV2

# Criar executor
executor = ExecutorV2()
executor.inicializar()

# Otimizar pedidos
solucao = executor.otimizar_pedidos(pedidos)

# Verificar resultados
print(f"Pedidos atendidos: {solucao.pedidos_atendidos}")
print(f"Taxa de sucesso: {solucao.taxa_sucesso}%")
```

7.2 Parametros de Configuracao

- **timeout_segundos:** Tempo maximo para resolucao (padrao: 600s)
- **inicio_jornada:** Horario de inicio da jornada de trabalho

8 Conclusao

O Otimizador PL v2.0 oferece uma abordagem matematicamente fundamentada para o problema de scheduling de producao, utilizando tecnicas de otimizacao combinatoria para maximizar o atendimento de pedidos respeitando restricoes de recursos e prazos.

A arquitetura modular permite facil extensao para novos modos de operacao (como o modo FLEXIVEL planejado para fases futuras) e integracao com o sistema de menu existente.